DOCUMENT / RESOLUTION / AUTHORITY / EVIDENCE

XXE Injection Explained

Understand XML, prove parser behavior, and control resource access

RESOLUTION

What resource is requested?

AUTHORITY

What may the worker access?

Understand the reader behind the document.

Find the weakness. Prove the effect. Verify the repair.

Learn through visual models and bounded local practice.

Haroon Shahid

First edition / Version 1.0 / 2 November 2026

Before you begin

XXE Injection Explained

Understand XML, prove parser behavior, and control resource access

By Haroon Shahid. First edition, version 1.0. 2 November 2026. Prepared with AI assistance.

Copyright 2026 Haroon Shahid. Personal learning license for the purchased publication; no resale rights or public redistribution of paid content. Original lab code is separately MIT licensed. The free preview may be shared unchanged with attribution.

Use only systems and accounts you own or are explicitly authorized to assess. This self-study kit does not certify expertise or guarantee findings, professional results, or income. All narrative services and review cases are fictional or explicitly identified source-based advisory analysis.

Fern Resolver Observatory uses real Expat parsing over finite original fixtures. Weak profiles explicitly add a custom resolver bounded to run-owned dummy files and fixed loopback HTTP. Default Expat does not fetch external URLs. The queue and profile controls are privileged in-memory teaching fixtures. They are not production services or arbitrary-input sanitizers.

An optional lxml bench runs fixed small examples for entity handling, XInclude, schema imports, and XSLT compilation/runtime resource access. It needs a separate lxml installation. The validation report names executed versions and checks. Other languages, frameworks, identity protocols, converters, cloud, containers, concurrency, and hard resource isolation are review guidance requiring native or deployment testing.

PDF code can soft-wrap for page fit. The offline HTML and editable source preserve original lines. On narrow screens, wide diagrams and tables scroll within their own containers. Vector diagrams have text labels and descriptions; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package includes all 64 chapters, answered reviews, the offline reading edition, practice workbooks, reference sheets, and the local Fern Resolver Observatory lab.

Contents

Before you begin 2
Contents 3
Introduction 4
Learning roadmap 5
01. The document is data. The process has power. 7
02. Bytes, text, encodings, and transport 9
03. Read XML without guessing 11
Sources and further reading 13

Introduction

A document contains a name. A parser reads it. Somewhere along the way, that name may become a file read or a request made by a server. Which step granted that power?

This book helps you answer that question. You do not need previous XML, programming, or security knowledge. We first build the ideas behind the format: bytes, markup, structure, namespaces, declarations, entities, and parsing. Only then do we investigate external resource access.

One service, many decisions

Fern Imports is fictional. Mira maintains its document pipeline. Customers should be able to submit book and order details. They should not be able to select private files, internal services, or unrestricted processing rules.

Fern Resolver Observatory is the local workshop. Its parser events, dummy files, and loopback HTTP requests are real. Its finite sample registry, profile switches, and manual queue are deliberate teaching controls. They make behavior visible without providing a general file reader or network scanner.

Four questions throughout

1. Document: what does the sender control, in which representation?

2. Resolution: which consumer interprets a reference, using which settings and context?

3. Authority: which process or service can access which resources for this task?

4. Evidence: which operations and results were observed, with which controls and limits?

Each chapter contains a concept diagram, a four-question revision trace, an analogy, examples, a summary, and three checks. Explained answers let you inspect the reasoning. Prediction exercises, guided local missions, synthetic review cases, and a worked capstone turn reading into practice.

A precise model travels further

A namespace URL is not an automatic request. An error does not prove disclosure. A blocked network connection does not prove local files are protected. A strict first reader does not configure a later transformer. These distinctions are the practical core of the book.

The source catalog identifies primary documentation and retrieval limits. The validation report separates executed experiments from native and deployment guidance. No single book can certify expertise or cover every future parser. The useful goal is a repeatable method for understanding an unfamiliar one.

Begin with the ordinary sample. Predict before running. Explain every result in your own words. When a prediction fails, follow the events rather than collect more unexplained examples. The most valuable habit is asking who interpreted the reference and why that component was allowed to act on it.

Learning roadmap

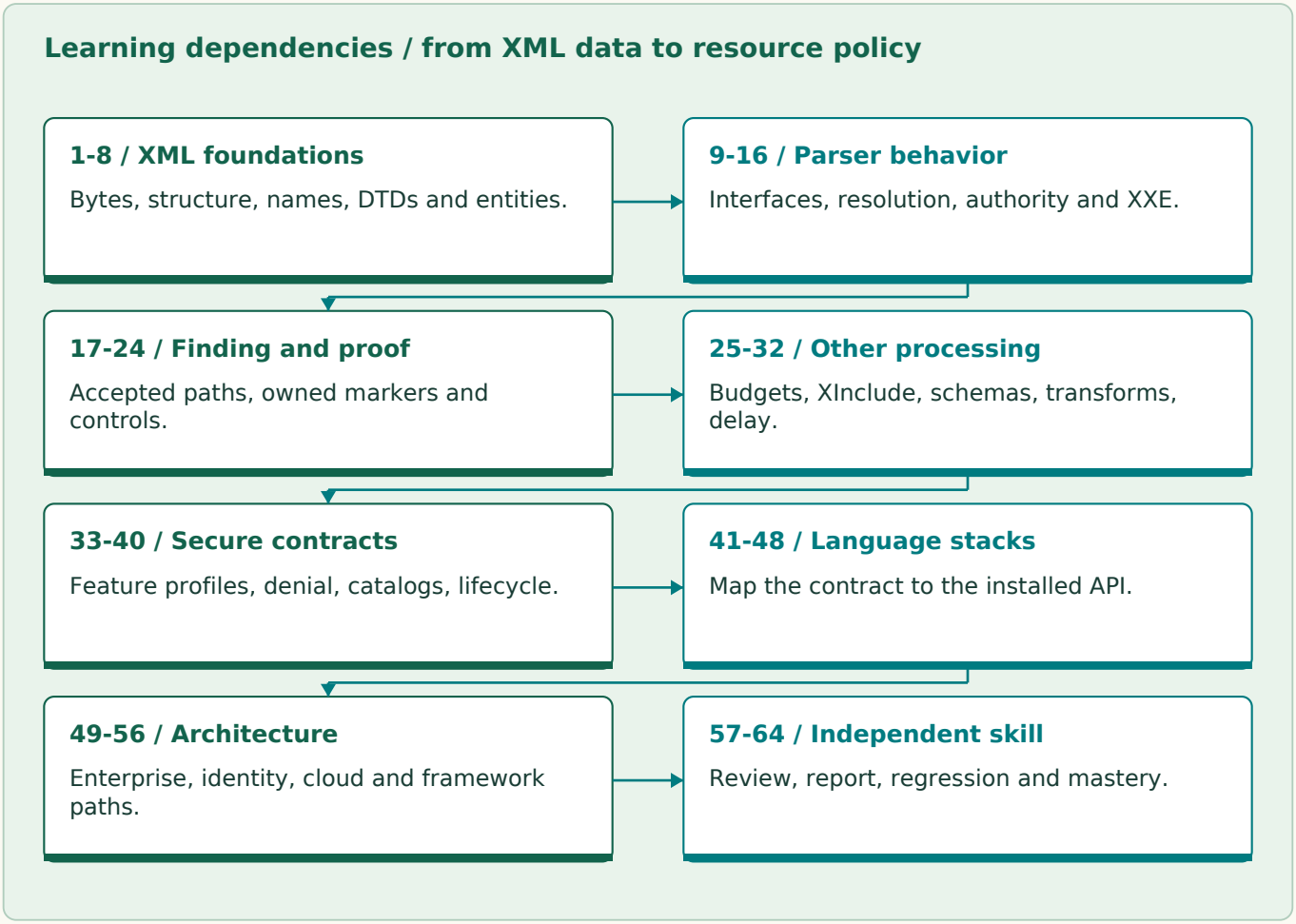
Choose a starting route

- New learner: Chapters 1-8, then the first four missions. Build XML vocabulary before examining resource access.

- Authorized tester: foundations, Chapters 9-24, then 57-60. Produce a bounded paired proof with honest impact limits.

- Developer: foundations, Chapters 25-48, then the relevant integration chapter. Implement a small feature/resource/budget contract.

- Architect: foundations, Chapters 32-40 and 49-62. Map consumers, identities, dependencies, tenants, jobs, and output publication.



Read left to right within each stage, then follow the connector to the next row. Each later stage uses the earlier mental model.

Eight stages, one connected model

Stage	Chapters	Demonstrate before moving on
Read XML	1-8	Explain bytes, elements, names, DTDs, entities, syntax and validity
Follow parsers	9-16	Trace interleaved resolution, context, authority and mechanism
Find and prove	17-24	Establish a healthy path and bounded file/request evidence
Separate consumers	25-32	Distinguish budgets, XInclude, schemas, transforms and delayed use
Design contracts	33-40	Specify features, denial, catalogs, limits and lifecycle ownership
Apply APIs	41-48	Map the contract to the actual language, entry point and version
Review architecture	49-56	Reason about enterprise, identity, converters, cloud and frameworks
Transfer the skill	57-64	Report, retest, assess incidents and analyze an unfamiliar path

Practice with a purpose

First predict literal text, character references and namespaces. Compare internal and external entities only after learning their context. Add file substitution, local requests and DTD chains. Then inspect failure timing, catalogs, finite budgets and queued work. Optional native missions exercise lxml's separate consumers.

The review clinic contains worked fictional designs, not additional deployed services. API recipes are acceptance contracts requiring the actual runtime. Use the claim ledger to distinguish locally observed, documentation-supported, inferred and unknown behavior.

Retain the model

The next day, explain one diagram without its caption. A week later, repeat predictions you missed and explain a new counterexample. After a month, solve the capstone without its answer. Revisit a prerequisite when you cannot explain a result.

Final progression

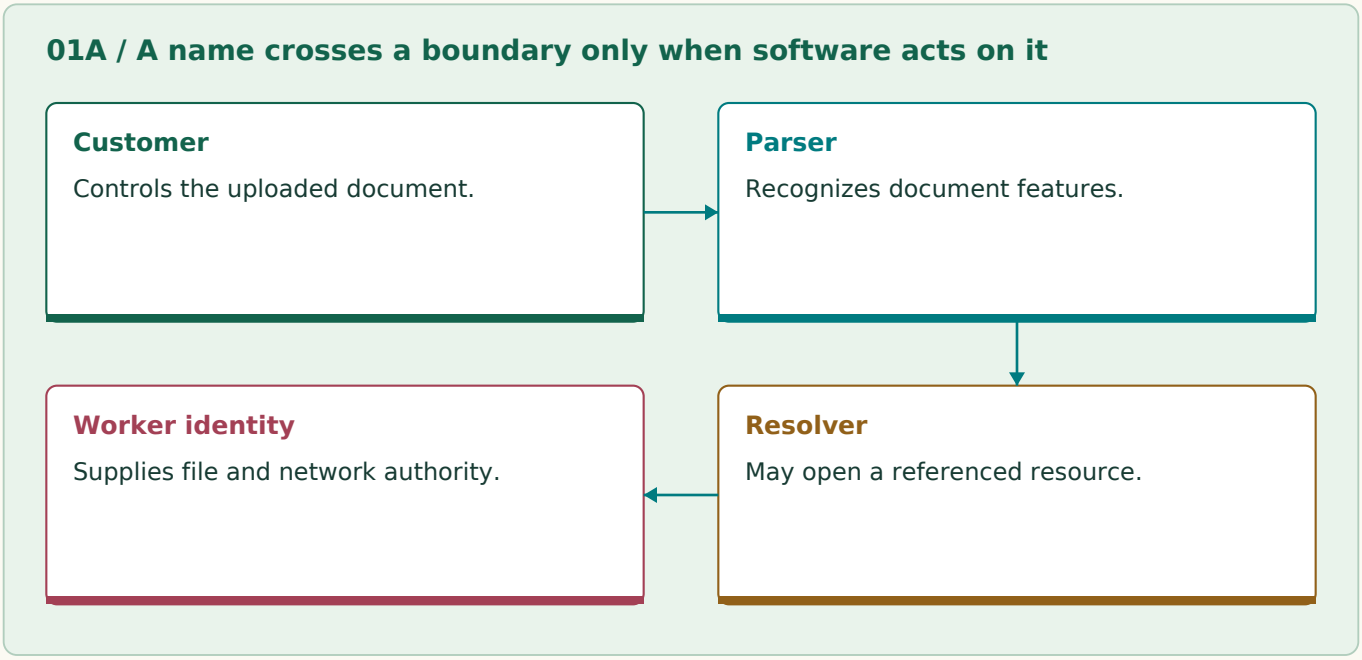
Collect four artifacts: an accurate consumer map, a controlled proof, a repaired contract with acceptance checks, and an unfamiliar-case review. These demonstrate reasoning. A reading streak, page count or quiz score alone does not establish professional expertise.

I. READ XML WITH CONFIDENCE

01. The document is data. The process has power.

Learn to: separate document content from the authority of its reader.

Think of it this way: A letter can name a locked room. The person reading the letter still decides whether to enter it.



Content, interpretation, and resource access are different facts.

Start with a small story

Mira runs **Fern Imports**, our fictional document service. A customer sends a list of books. The service reads the list, checks it, and shows a summary. The customer should control book titles. The customer should not choose which files the service opens or which servers it contacts.

Data is information: a title, number, date, or document. A **format** is a shared way to write that information. A text file contains bytes, which are small numeric units. Software interprets those bytes using a format. XML is one such format. We will build it from the beginning.

The **application** is the service's code. A **process** is a running program. Its operating-system identity may be able to read files or use the network. A **parser** is code that reads a format and produces structure the application can use. It runs with authority supplied by its environment. The document does not need its own login to benefit from that authority.

Follow three separate facts

A document may contain a resource name. The parser may recognize it as a reference. A resolver may then open the resource. These are different events. Seeing a name in a document proves none of the later events by itself.

For example, `<title>guide.txt</title>` normally holds ordinary text. It does not tell a basic XML parser to open that file. A feature that treats a name as an external reference changes the

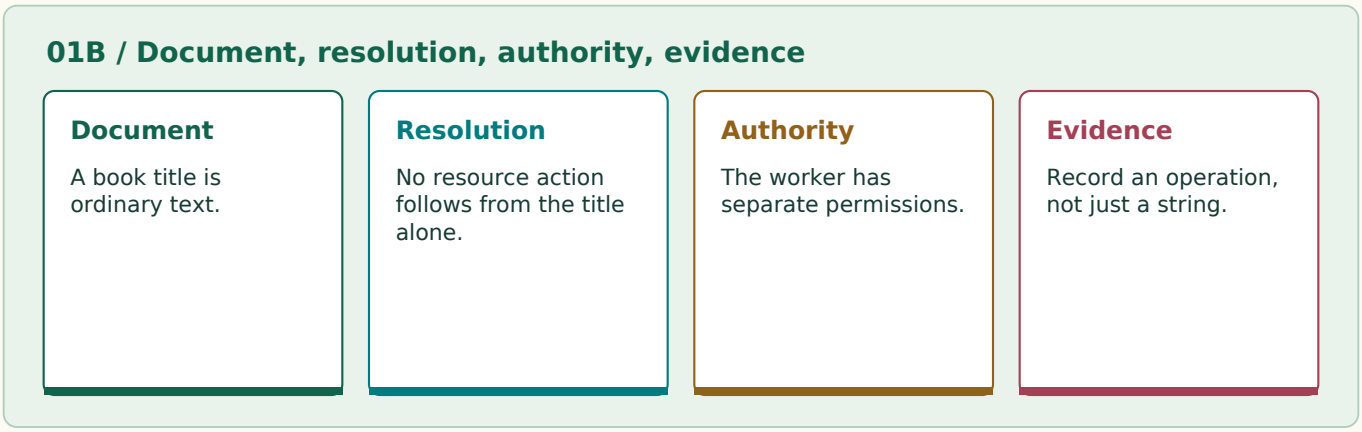
situation. That feature, its settings, and the resolver become part of the security boundary.

A **trust boundary** is a place where one party's control meets another party's power. Here the customer's document meets the import worker. Security requires the worker to keep its power within the intended task.

Your first useful habit

Ask: who controls this value, who interprets it, and who performs any resulting operation? Keep those answers separate in a notebook. This habit will later explain file disclosure, server-side requests, blind evidence, and secure catalogs.

Try it: name one file a document service needs and one it should never need. Explain the difference using the task, not the file extension. You already have the beginning of a resource policy.



Key takeaway. Content, interpretation, and resource access are different facts.

Check your understanding

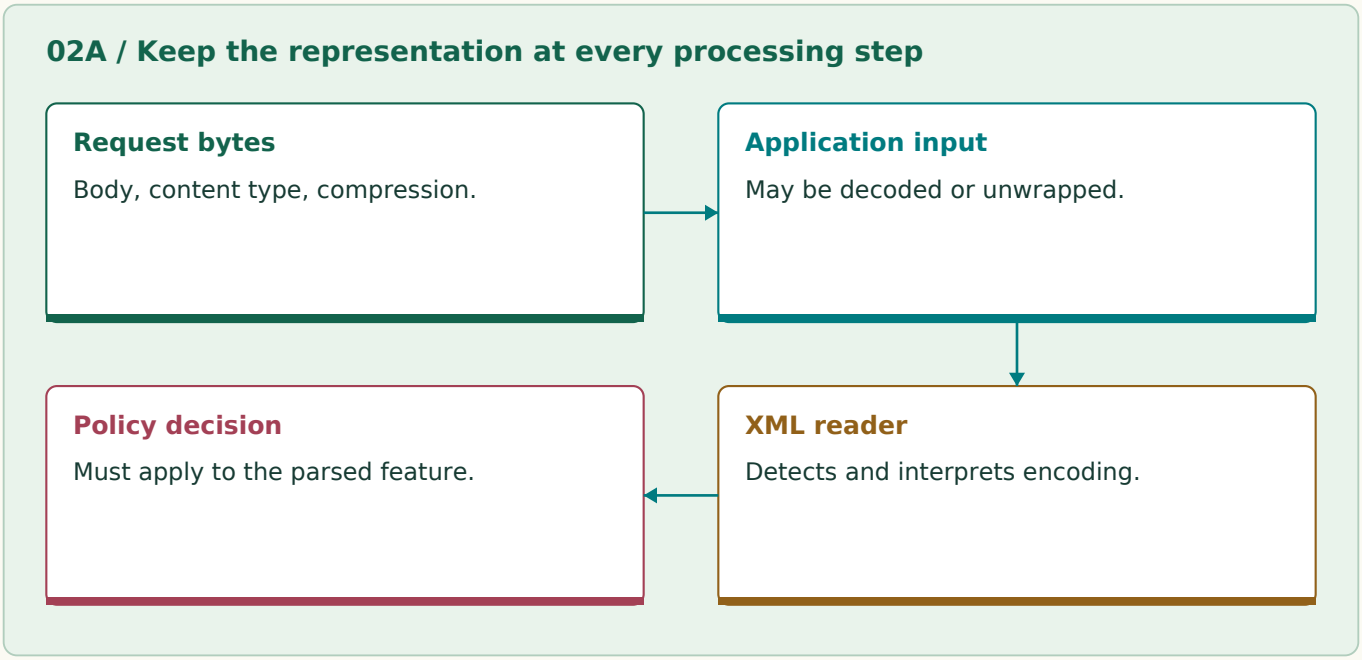
1.1 Does a filename in a title prove a file read?	**1.2** Whose permissions matter during parsing?	**1.3** What is the first boundary to draw?

Answer explanations are in the full ebook. Sources: R06, R08, R09

02. Bytes, text, encodings, and transport

Learn to: follow the actual bytes reaching a parser.

Think of it this way: A message can use different alphabets while keeping the same meaning. The reader must know which alphabet was used.



Test the representation the actual parser receives.

What is an encoding?

An **encoding** maps characters to bytes. A character is something we read, such as A or <. UTF-8 and UTF-16 are encodings that XML processors are required to support. The same visible text has different byte patterns in them. A **byte order mark**, or BOM, can help identify an encoding.

An XML declaration can state an encoding. It is not a DTD, an entity declaration, or a request to load a resource. Here is a small complete document:

```
<?xml version="1.0" encoding="UTF-8"?>
<book><title>Fern Guide</title></book>
```

The declaration describes the document. The root element is book. We will unpack elements next.

The trip to the application

A browser or client sends an HTTP request. A request has a method, headers, and sometimes a body. The body may contain XML directly. It may instead contain a file inside a multipart upload or XML text inside another data format. A header such as `Content-Type: application/xml` describes the representation. It does not prove that the server uses an XML parser.

HTTPS protects the network connection. It does not make the uploaded XML trustworthy. An authenticated customer can still send unsafe content.

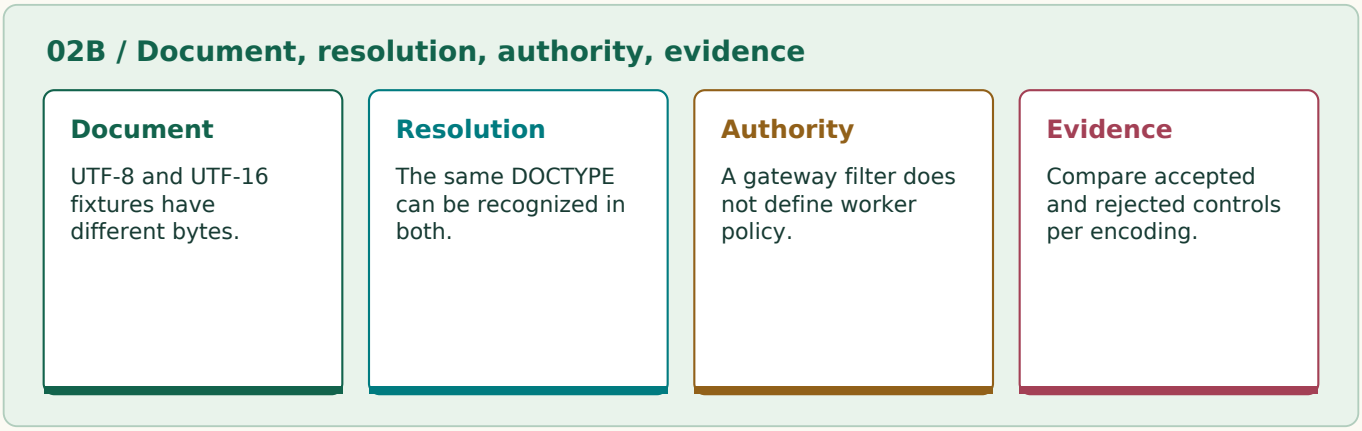
Different layers may decompress, decode, or convert the body before parsing. Record each step. If a gateway scans the raw body for ASCII bytes while a worker reads UTF-16, the two layers are not

examining the same representation. A successful text scan is not a parser security policy.

A controlled comparison

In the lab, one fixture is encoded as UTF-8 and another as UTF-16. Both contain the same document feature. Predict whether the parser-level policy will reject both. Then compare this with a deliberately weak byte filter. This is a small representation experiment, not permission to guess arbitrary encodings against public services.

Common mistake: editing the encoding label without changing the bytes does not create a sound test. The document may simply become unreadable. Keep an accepted ordinary document in each encoding as a control. Save exact bytes and their hash when evidence depends on representation.



Key takeaway. Test the representation the actual parser receives.

Check your understanding

2.1 Does HTTPS prevent XXE?	**2.2** Does application/xml prove XML parsing?	**2.3** Why keep ordinary UTF-16 input as a control?

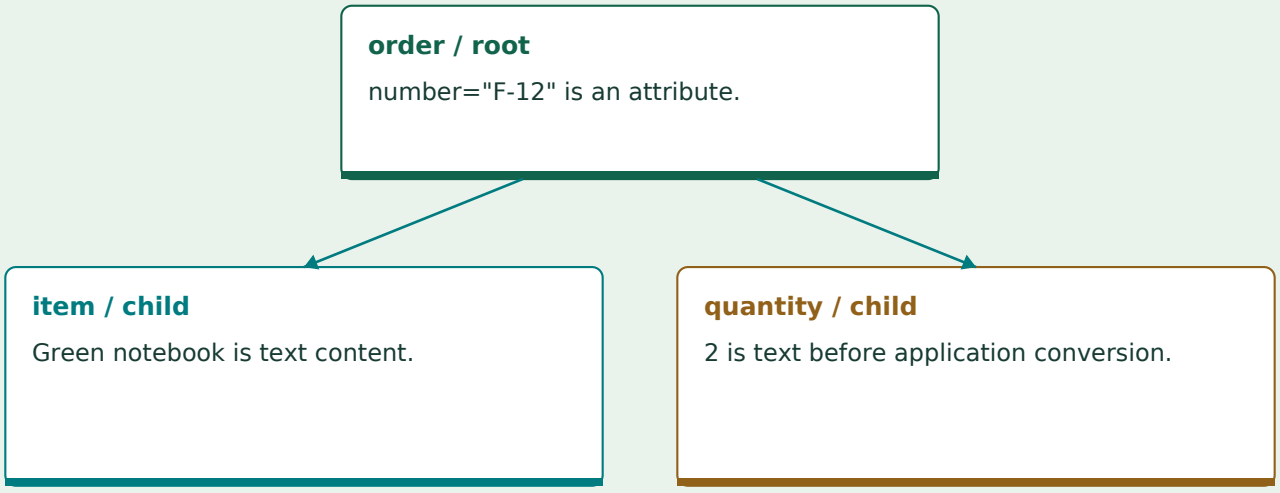
Answer explanations are in the full ebook. Sources: R04, R10, R60

03. Read XML without guessing

Learn to: recognize elements, attributes, nesting, and literal text.

Think of it this way: Markup is a set of labeled boxes. Each label says where the contents belong.

03A / A small XML tree has labels, contents, and relationships



XML syntax creates structure; application logic assigns business meaning.

Markup gives text structure

A **markup language** uses written markers to describe parts of a document. XML means Extensible Markup Language. Its 1998 first recommendation grew from SGML. It provided a simpler common syntax for structured documents. XML lets an application define its own vocabulary. It does not give every tag a built-in action.

```
<order number="F-12">
  <item>Green notebook</item>
  <quantity>2</quantity>
</order>
```

An **element** includes its start tag, contents, and end tag. `order` is the root element, the one outer element. An **attribute** is a name and quoted value in a start tag. Here `number` is an attribute. A child element sits inside another element. The text 2 is character data; the application may later convert it to a number.

Tags must nest correctly. <a> is not well-formed XML. Case matters: `Item` and `item` are different names. <item/> is an empty element. Comments are notes in `<!-- ... -->`. They are not ordinary element text. Software should use the parser's model instead of extracting fields with a few string splits.

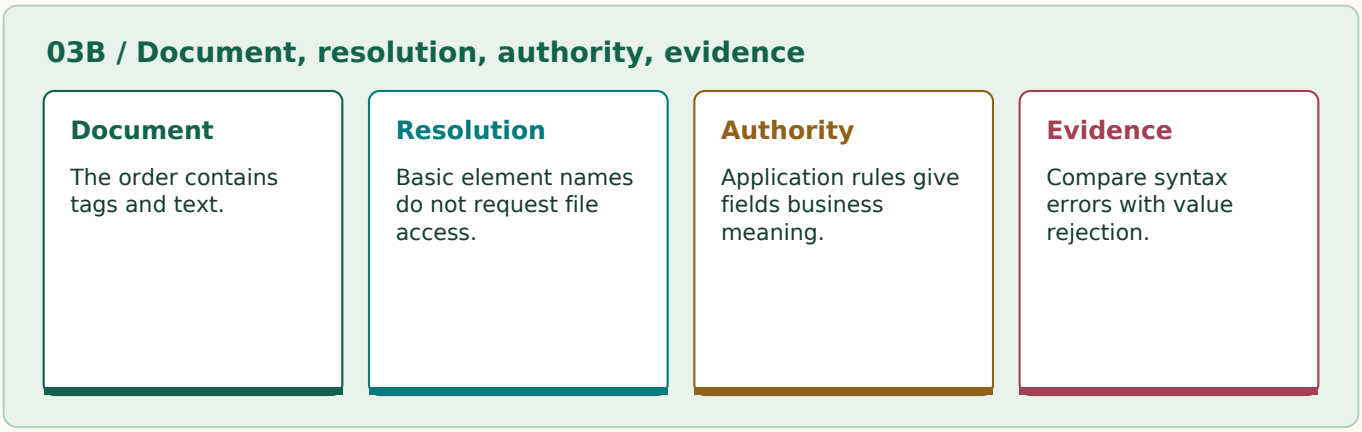
HTML and XML are different contracts

HTML defines a web document vocabulary and browser behavior. XML defines syntax for application vocabularies. An XML tag named `script` does not, by itself, make a server execute JavaScript. SVG has its own vocabulary and browser behavior, but that is a later layer. Calling every XML processing problem XSS or XXE hides the actual mechanism.

Your first prediction

Change `<quantity>2</quantity>` to `<quantity>two</quantity>`. Is the result well-formed? Yes: the XML syntax can still be correct. Is it acceptable for an order? That depends on validation and application rules, which we teach in Chapter 6.

Change the closing tag to `</Quantity>`. Now parsing should fail because it no longer matches. Distinguish a syntax error from a business-rule rejection. A raw error alone does not prove an external resource was accessed.



Key takeaway. XML syntax creates structure; application logic assigns business meaning.

Check your understanding

3.1 Is quantity two a syntax error?	**3.2** Does a tag named script always execute?	**3.3** Why does a mismatched closing tag matter?

Answer explanations are in the full ebook. Sources: R01, R03, R36

Sources and further reading

Research cutoff: 2 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

R01 - XML 1.0, Fifth Edition

Syntax, DTDs, parsed entities, contexts and well-formedness. Consulted relevant sections; core XML examples are original.

https://www.w3.org/TR/REC-xml/

R03 - XML Information Set

Conceptual document information model. Consulted; not an exact-byte preservation guarantee.

https://www.w3.org/TR/xml-infoset/

R04 - RFC 7303: XML Media Types

Media types and XML representation/encoding context. Consulted search/relevant text.

https://www.rfc-editor.org/rfc/rfc7303

R06 - CWE-611: XML External Entity Reference

Weakness classification and mechanism. Consulted. The fictional service is not a reported victim.

https://cwe.mitre.org/data/definitions/611.html

R08 - Expat XML Security

Default no-fetch behavior, custom external handlers and resource-consumption fixes. Consulted. The core lab explicitly adds its weak handler.

https://libexpat.github.io/doc/xml-security/

R09 - Python XML security

Native dependency and module-specific caveats. Consulted current and versioned documentation; check deployed Expat separately.

https://docs.python.org/3/library/xml.html#xml-security

R10 - Python Expat interface

Handlers, external child parsers, parameter entities and byte input. Consulted and exercised by the core lab.

https://docs.python.org/3/library/pyexpat.html

R36 - SVG 2 specification

XML-based graphics vocabulary and implementation context. Consulted overview. SVG XML parsing is not native rendering or browser execution.

https://www.w3.org/TR/SVG2/

R60 - RFC 9110: HTTP Semantics

Requests, representations and HTTP result context. Consulted relevant sections.

https://www.rfc-editor.org/rfc/rfc9110.html